

1:50-2:15pm, Monday, February 9, 2024.

1. **Big-Oh notation.** Simplify each function, and find all pairs where $f(n) \in \Theta(g(n))$.

$$21n + \sqrt{n \log^2 n}, \quad 2^{\log_2 n + 3}, \quad (n + \sqrt{n})(\log^2 n - \sqrt{\log n}), \quad \frac{(n \log n)^2}{n + \log n}.$$

Answer. $f_1(n) = 21n + \sqrt{n \log^2 n} = \Theta(n)$,
 $f_2(n) = 2^{\log_2 n + 3} = 2^{\log_2 n} \cdot 2^3 = 8n = \Theta(n)$,
 $f_3(n) = (n + \sqrt{n})(\log^2 n - \sqrt{\log n}) = \Theta(n \log^2 n)$,
 $f_4(n) = \frac{(n \log n)^2}{n + \log n} = \Theta\left(\frac{n^2 \log^2 n}{n}\right) = \Theta(n \log^2 n)$.

We see that the pairs (f_1, f_2) and (f_3, f_4) have the same growth rates.

2. Determine the asymptotic running time of the algorithm below in terms of n .

```

begin
  i ← 2
  while i ≤ n do
    j ← 1
    while j ≤ i do
      print "abc"
      j ← j + 1
    i ← 2 * i

```

Answer. The outer loop runs $O(\log n)$ times, with values $i = 2, 4, 8, \dots, 2^{\log n}$, or $i = 2^k$ for $k = 1, 2, 3, \dots, \log n$. The inner loop runs $i = 2^k$ times (j goes from 1 to i with increments of 1). We see that the two loops are not independent. So the overall running time is the summation of the inner loops:

$$2 + 4 + 8 + \dots + 2^{\log n} = \sum_{k=1}^{\log n} 2^k = O(2^{\log n}) = O(n).$$

(TURN THE PAGE)

3. In the *Towers of Hanoi* puzzle, we want to move n disks from pin 1 to pin 3. In class, we discussed a recursive algorithm that achieves this in $T(n) = 2^n - 1$ steps. Describe this algorithm (either in words or with a pseudo-code).

Answer.

Recursively move $n - 1$ disks from pin 1 to pin 2 (using pin 3 as temporary pin);

Move the largest disk from pin 1 to pin 3;

Recursively move $n - 1$ disks from pin 2 to pin 3 (using pin 1 as temporary pin).

4. In class, we discussed a recursive algorithm for $\text{SELECTION}(A[1..n], k)$ with recurrence relation $T(n) = T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + O(n)$. What recurrence would we get if we modify the algorithm as follows. (Explain each term of the recurrence in your answer.)

- (1) Divide the array $A[1..n]$ into subarrays of size 3; find the median of each subarray; and let $B[1..\frac{n}{3}]$ be the array of these medians;
- (2) $p \leftarrow \text{SELECTION}(B[1..\frac{n}{3}], \frac{n}{9})$;
- (3) $q \leftarrow \text{SELECTION}(B[1..\frac{n}{3}], \frac{2n}{9})$;
- (4) Partition $A[1..n]$ into **three** arrays using **two** pivots, p and q ;
- (5) Recurse on the part that contains the element of rank k in $A[1..n]$.

Answer. Line (1) takes $O(n)$ time. The recursive calls in lines (2) and (3) each take $T(n/3)$ time. Partition in line (4) takes $O(n)$ time. In line (5), we recurse on a subarray of size m in time $T(m)$. So far, we have

$$T(n) = 2T\left(\frac{n}{3}\right) + T(m) + O(n).$$

It remains to give an upper bound for m . Suppose x is the item of rank k . We distinguish between three cases: Case 1: If $x \leq p$, then we can eliminate the $2n/9$ entries in $B[(\frac{n}{9} + 1)..\frac{n}{3}]$ that are larger than p , and $2n/9$ additional entries (one in each group); so $m \leq n - 2n/9 - 2n/9 = 5n/9$. Case 2: If $p < x \leq q$, then we can eliminate $n/9$ entries in $B[1..\frac{n}{9}]$ that are less than p and $n/9$ the entries $B[(\frac{2n}{9} + 1)..\frac{n}{3}]$ that are larger than q , as well as $2n/9$ additional entries (one in each group); so $m \leq n - 2n/9 - 2n/9 = 5n/9$ again. Case 3, where $q < x$, is symmetric to Case 1, so $m \leq 5n/9$ in all three cases. Overall,

$$T(n) = 2T\left(\frac{n}{3}\right) + T\left(\frac{5n}{9}\right) + O(n).$$